

Strategy Design Pattern (Behavioral)

Encapsulate each algorithm behind a common interface so that the algorithm used by a client can be swapped at runtime without modifying the client.

Intent

The Strategy pattern solves the problem of hard-coding a single algorithm inside a class, which forces a code change every time a different algorithm is needed. It defines a family of algorithms, encapsulates each one in its own class, and makes them interchangeable via a shared interface. The object that uses the algorithm (the *context*) delegates the work to whichever strategy it holds, and neither the context nor the caller needs to know the implementation details.

Real-world analogy

Think of a pocket calculator with a mode dial. The display, the number buttons, and the equals key never change — that is the **context**. What changes is the *operation* selected: add, subtract, multiply, or divide. Each mode is an independent, interchangeable strategy. Rotating the dial (injecting a different strategy) does not require rewiring the calculator; it simply changes which algorithm the fixed infrastructure delegates to when the equals key is pressed.

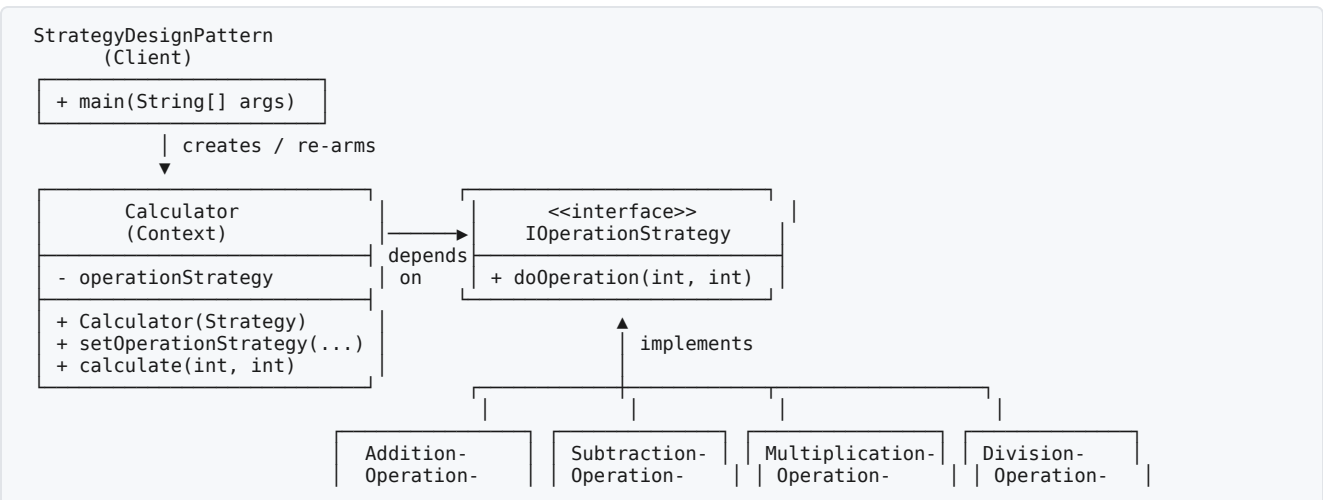
How the pattern works

When `main()` constructs a `Calculator` it passes in an `IOperationStrategy`. The `Calculator` stores that reference in a field. Every call to `calculate(x, y)` is forwarded to `operationStrategy.doOperation(x, y)`. The `Calculator` never knows — or cares — whether it is adding, subtracting, multiplying, or dividing. Any class that implements `IOperationStrategy` can be plugged in without touching `Calculator` at all. Beyond construction-time injection, `Calculator` also exposes `setOperationStrategy(...)`, so the very same context instance can be re-armed with a different algorithm at runtime — the client in this project does exactly that, swapping the strategy four times on one `Calculator` object to perform addition, subtraction, multiplication, and division in sequence.

The four structural roles are:

Role	This project
Strategy (interface)	<code>IOperationStrategy</code>
Concrete Strategy	<code>AdditionOperationStrategy</code> , <code>SubtractionOperationStrategy</code> , <code>MultiplicationOperationStrategy</code> , <code>DivisionOperationStrategy</code>
Context	<code>Calculator</code>
Client	<code>StrategyDesignPattern.main()</code>

UML class diagram (ASCII)



Strategy	Strategy	Strategy	Strategy
doOperation: return x + y	doOperation: return x - y	doOperation: return x * y	doOperation: return x / y

The players

- **IOperationStrategy** (interface — Strategy role) — declares the single contract method `doOperation(int x, int y)` that every algorithm must honour. The existence of this interface is what makes all concrete strategies interchangeable from the context's point of view.
- **AdditionOperationStrategy** (Concrete Strategy) — implements `IOperationStrategy` by returning `x + y`.
- **SubtractionOperationStrategy** (Concrete Strategy) — implements `IOperationStrategy` by returning `x - y`.
- **MultiplicationOperationStrategy** (Concrete Strategy) — implements `IOperationStrategy` by returning `x * y`.
- **DivisionOperationStrategy** (Concrete Strategy) — implements `IOperationStrategy` by returning `x / y` using integer division.
- **Calculator** (Context) — stores an `IOperationStrategy` reference and delegates all arithmetic to it. It is entirely decoupled from every concrete strategy class. Besides constructor injection, it exposes `setOperationStrategy(...)`, which lets the same instance be re-armed with a different algorithm at runtime.
- **StrategyDesignPattern** (Client / entry point) — starts the Spring Boot application and exercises the pattern by wiring `AdditionOperationStrategy` into a `Calculator`, printing the result of `calculate(5, 20)`, and then reusing the same `Calculator` instance three more times — calling `setOperationStrategy` with `SubtractionOperationStrategy`, `MultiplicationOperationStrategy`, and `DivisionOperationStrategy` in turn and printing each result — to demonstrate that the strategy can be swapped at runtime without replacing the context.

Code walkthrough — every line explained

IOperationStrategy.java

```
package com.design.patterns.strategy;

public interface IOperationStrategy {
    public int doOperation(int x, int y);
}
```

Line by line:

- `package com.design.patterns.strategy;` — Declares that this compilation unit belongs to the `com.design.patterns.strategy` package. It is placed in its own package (separate from the concrete implementations which live in `.concreteStrategy` and the context which lives in `.context`) so that the strategy interface layer is isolated: any code that depends only on the abstraction imports from this package and never needs to know the concrete sub-package exists.
- *(blank line)* — Separates the package declaration from the type declaration, following standard Java style.
- `public interface IOperationStrategy {` — Declares a public interface named `IOperationStrategy`. Using an interface rather than an abstract class is the canonical choice for the Strategy pattern: it enforces zero shared state, allows implementing classes to freely extend any superclass they choose, and makes the contract maximally explicit with no hidden behaviour. The opening `{` begins the interface body.
- `public int doOperation(int x, int y);` — Declares the single abstract method that every concrete strategy must implement. `public` is redundant on an interface method (all interface methods are implicitly public) but is written explicitly here for clarity. It returns `int` so that the caller receives the arithmetic result directly without wrapper types or output parameters. The parameters `x` and `y` are named generically to keep the interface neutral — it does not imply addition, subtraction, or any specific operation. The `;` ends the method declaration without a body, as is required for abstract interface methods in Java 8 (the version targeted by this project's parent POM).
- *(blank line)* — Trailing blank line inside the interface body before the closing brace.
- `}` — Closes the `IOperationStrategy` interface body.

AdditionOperationStrategy.java

```
package com.design.patterns.strategy.concreteStrategy;

import com.design.patterns.strategy.IOperationStrategy;

public class AdditionOperationStrategy implements IOperationStrategy {

    @Override
    public int doOperation(int x, int y) {
        return x + y;
    }
}
```

Line by line:

- `package com.design.patterns.strategy.concreteStrategy;` — Places this class in the `concreteStrategy` sub-package. Grouping all four concrete strategies here separates "what algorithms exist" from "what the algorithm contract is" and from "who uses the algorithm". This mirrors a clean layering principle: strategy interface, concrete strategies, and context each occupy their own package.
- *(blank line)* — Separates the package declaration from the import block, per Java convention.
- `import com.design.patterns.strategy.IOperationStrategy;` — Brings the `IOperationStrategy` interface into scope so the `implements IOperationStrategy` clause compiles. It is a named single-type import rather than a wildcard (`strategy.*`) so that the dependency is explicit and readable at a glance.
- *(blank line)* — Separates the import block from the class declaration.
- `public class AdditionOperationStrategy implements IOperationStrategy {` — Declares a public class that fulfils the `IOperationStrategy` contract. `implements IOperationStrategy` is the mechanism that makes this class a drop-in replacement for any other strategy wherever an `IOperationStrategy` reference is required. The opening `{` begins the class body.
- *(blank line)* — Blank line inside the class for readability, separating the class header from the method.
- `@Override` — Annotation that tells the compiler this method must match a signature declared in a supertype. If the signature ever drifts from `IOperationStrategy.doOperation(int, int)` — for example, if the interface is refactored — the compiler reports an error rather than silently creating an unrelated new method. This is a correctness guard with no runtime cost.
- `public int doOperation(int x, int y) {` — Concrete implementation of the strategy method. `public` is required because the interface method is public and Java does not allow reducing visibility when overriding. `int` matches the interface return type exactly. The opening `{` begins the method body.
- `return x + y;` — The addition algorithm. Returns the integer sum of the two operands. This is the only line that distinguishes `AdditionOperationStrategy` from the three other concrete strategies — the entire point of the Strategy pattern is to isolate exactly this algorithmic difference into one place.
- `}` — Closes the `doOperation` method body.
- `}` — Closes the `AdditionOperationStrategy` class body.

SubtractionOperationStrategy.java

```
package com.design.patterns.strategy.concreteStrategy;

import com.design.patterns.strategy.IOperationStrategy;

public class SubtractionOperationStrategy implements IOperationStrategy {

    @Override
    public int doOperation(int x, int y) {
        return x - y;
    }
}
```

Line by line:

- `package com.design.patterns.strategy.concreteStrategy;` — Same `concreteStrategy` sub-package as all concrete strategies, grouping them together for the cohesion reason explained above.
- *(blank line)* — Standard Java separator between package declaration and imports.
- `import com.design.patterns.strategy.IOperationStrategy;` — Brings the `IOperationStrategy` interface into scope so the `implements` clause compiles.

- *(blank line)* — Separates the import block from the class declaration.
 - `public class SubtractionOperationStrategy implements IOperationStrategy {` — Declares the subtraction concrete strategy. The class name encodes both the operation (`Subtraction`) and the role (`IOperationStrategy`), making its purpose immediately readable in any code that instantiates it. The opening `{` begins the class body.
 - *(blank line)* — Blank line for readability inside the class body.
 - `@Override` — Compiler-enforced override guard, same purpose as in `AdditionOperationStrategy`.
 - `public int doOperation(int x, int y) {` — Concrete implementation with the same visibility and return type as the interface declaration. Opening `{` begins the method body.
 - `return x - y;` — The subtraction algorithm. Returns `x` minus `y`. The order of operands matters: `x - y` not `y - x`. The caller provides operands in a meaningful order and this implementation honours that order consistently with the standard mathematical convention for subtraction.
 - `}` — Closes the `doOperation` method body.
 - `}` — Closes the `SubtractionOperationStrategy` class body.
-

MultiplicationOperationStrategy.java

```
package com.design.patterns.strategy.concreteStrategy;
import com.design.patterns.strategy.IOperationStrategy;
public class MultiplicationOperationStrategy implements IOperationStrategy {
    @Override
    public int doOperation(int x, int y) {
        return x * y;
    }
}
```

Line by line:

- `package com.design.patterns.strategy.concreteStrategy;` — Same concrete-strategy sub-package, same cohesion reasoning as the other concrete strategies.
 - *(blank line)* — Standard Java separator.
 - `import com.design.patterns.strategy.IOperationStrategy;` — Pulls the strategy interface type into this compilation unit's scope so the `implements` clause is resolvable.
 - *(blank line)* — Separates imports from the class declaration.
 - `public class MultiplicationOperationStrategy implements IOperationStrategy {` — Declares the multiplication concrete strategy. `implements IOperationStrategy` binds it to the same contract as all other strategies, making it substitutable anywhere a `Calculator` (or any other context) expects an `IOperationStrategy` reference. The opening `{` begins the class body.
 - *(blank line)* — Whitespace for readability.
 - `@Override` — Compiler-enforced override guard ensuring the method signature stays consistent with the interface.
 - `public int doOperation(int x, int y) {` — Concrete implementation with the signature matching the interface. Opening `{` begins the method body.
 - `return x * y;` — The multiplication algorithm. Returns the integer product of the two operands. Like all other strategies the entire algorithm is one expression, demonstrating how cleanly the Strategy pattern isolates each variant into a single focused line.
 - `}` — Closes the `doOperation` method body.
 - `}` — Closes the `MultiplicationOperationStrategy` class body.
-

DivisionOperationStrategy.java

```
package com.design.patterns.strategy.concreteStrategy;
import com.design.patterns.strategy.IOperationStrategy;
public class DivisionOperationStrategy implements IOperationStrategy {
```

```

@Override
public int doOperation(int x, int y) {
    return x / y;
}
}

```

Line by line:

- `package com.design.patterns.strategy.concreteStrategy;` — Same concrete-strategy sub-package as the three other concrete strategies.
- *(blank line)* — Standard Java separator between package declaration and import block.
- `import com.design.patterns.strategy.IOperationStrategy;` — Makes `IOperationStrategy` visible in this compilation unit so the `implements` clause resolves correctly.
- *(blank line)* — Separates the import block from the class declaration.
- `public class DivisionOperationStrategy implements IOperationStrategy {` — Declares the division concrete strategy. It follows the identical structural pattern as every other concrete strategy in this project. The opening `{` begins the class body.
- *(blank line)* — Whitespace for readability inside the class.
- `@Override` — Ensures the method truly overrides the interface method and that any future signature mismatch is caught at compile time.
- `public int doOperation(int x, int y) {` — Concrete implementation. Same visibility and return type as the interface. Opening `{` begins the method body.
- `return x / y;` — The integer division algorithm. Returns the quotient of `x` divided by `y` using Java's integer division, which truncates any fractional part toward zero. This implementation does not guard against `y == 0`; that is a deliberate simplification in this educational example to keep attention on the pattern structure rather than defensive programming edge cases.
- `}` — Closes the `doOperation` method body.
- `}` — Closes the `DivisionOperationStrategy` class body.

Calculator.java

```

package com.design.patterns.context;

import com.design.patterns.strategy.IOperationStrategy;

public class Calculator {

    private IOperationStrategy operationStrategy;

    public Calculator(IOperationStrategy operationStrategy) {
        this.operationStrategy = operationStrategy;
    }

    public void setOperationStrategy(IOperationStrategy operationStrategy) {
        this.operationStrategy = operationStrategy;
    }

    public int calculate(int x, int y) {
        return operationStrategy.doOperation(x, y);
    }

}

```

Line by line:

- `package com.design.patterns.context;` — Places the `Calculator` class in the `context` sub-package. Naming the package `context` directly mirrors GoF terminology: the *context* is the object that holds a strategy reference and delegates work to it. Keeping it in its own package reinforces the separation between the context role and the strategy role — any code can import the context without importing the strategies, and vice versa.
- *(blank line)* — Standard Java separator between the package declaration and the import block.
- `import com.design.patterns.strategy.IOperationStrategy;` — Brings the `IOperationStrategy` interface type into scope. The context depends only on the interface, not on any concrete strategy class. This is the core of the Dependency Inversion Principle at work: `Calculator` imports from the `strategy` package (the abstraction), never from `strategy.concreteStrategy` (the implementations). This import is the only cross-package dependency `Calculator` has.

- *(blank line)* — Separates the import block from the class declaration.
- `public class Calculator {` — Declares the context class. It is `public` so that the client (`StrategyDesignPattern`) in a different package can instantiate it. Opening `{` begins the class body.
- *(blank line)* — Blank line separating the class header from the field declaration, following standard Java formatting.
- `private IOperationStrategy operationStrategy;` — Declares an instance field that holds a reference to whichever strategy this calculator is currently configured with. It is typed as the *interface* (`IOperationStrategy`), not as any concrete class — this is the polymorphic dependency that makes the Strategy pattern work. It is `private` to enforce encapsulation: no external code can read the strategy directly or overwrite it except through the constructor or the setter below. This field is the single point through which the entire pattern's flexibility flows.
- *(blank line)* — Blank line separating the field declaration from the constructor.
- `public Calculator(IOperationStrategy operationStrategy) {` — Declares the constructor that accepts an `IOperationStrategy`. Using constructor injection (rather than relying solely on a setter, or instantiating a default strategy inside the constructor body) makes the dependency mandatory and visible: you cannot create a `Calculator` without supplying a strategy, preventing the field from ever being `null` due to a forgotten initialisation. The parameter shares its name with the field; inside the body `this.operationStrategy` disambiguates the field from the parameter. The opening `{` begins the constructor body.
- `this.operationStrategy = operationStrategy;` — Stores the injected strategy into the instance field. `this.operationStrategy` explicitly refers to the field declared above; the bare `operationStrategy` on the right-hand side refers to the constructor parameter, which Java resolves by scope (the nearer, parameter-level declaration wins when unqualified). This single assignment is the entire body of the constructor: all the context setup amounts to wiring the strategy reference.
- `}` — Closes the constructor body.
- *(blank line)* — Blank line separating the constructor from the setter.
- `public void setOperationStrategy(IOperationStrategy operationStrategy) {` — Declares a setter that lets callers replace the strategy on an already-constructed `Calculator`. This is what makes the pattern demonstrably *dynamic*: without this method, switching algorithms would require discarding the old `Calculator` and building a new one. With it, the same context instance persists across strategy changes — its identity does not change even though its behaviour does. The opening `{` begins the method body.
- `this.operationStrategy = operationStrategy;` — Overwrites the field with the newly supplied strategy, exactly mirroring the assignment in the constructor. Any `calculate(...)` call made after this line dispatches to the new strategy's `doOperation`; any call made before it dispatched to the old one. No other state in `Calculator` changes.
- `}` — Closes the `setOperationStrategy` method body.
- *(blank line)* — Blank line separating the setter from the `calculate` method.
- `public int calculate(int x, int y) {` — Declares the public API of the context. Callers ask the `Calculator` to compute a result; they pass two integer operands and receive the result. The method name `calculate` is intentionally generic — it does not say "add" or "multiply" because the actual operation depends on which strategy is currently held. This genericity is important: the same method name works regardless of which algorithm is plugged in, and regardless of whether that algorithm was set via the constructor or via `setOperationStrategy`. Opening `{` begins the method body.
- `return operationStrategy.doOperation(x, y);` — Delegates the computation entirely to whichever strategy the field currently references. The `Calculator` performs no arithmetic itself; it simply forwards the call. This single line is the heart of the Strategy pattern in the context: the context knows *that* an operation must happen but not *how* it happens. The result returned by the strategy is passed straight back to the caller without transformation.
- `}` — Closes the `calculate` method body.
- *(blank line)* — Trailing blank line before the class closing brace.
- `}` — Closes the `Calculator` class body.

StrategyDesignPattern.java

```
package com.design.patterns;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import com.design.patterns.context.Calculator;
import com.design.patterns.strategy.concreteStrategy.AdditionOperationStrategy;
```

```

import com.design.patterns.strategy.concreteStrategy.DivisionOperationStrategy;
import com.design.patterns.strategy.concreteStrategy.MultiplicationOperationStrategy;
import com.design.patterns.strategy.concreteStrategy.SubtractionOperationStrategy;

@SpringBootApplication
public class StrategyDesignPattern {

    public static void main(String[] args) {
        SpringApplication.run(StrategyDesignPattern.class, args);

        Calculator calculator = new Calculator(new AdditionOperationStrategy());
        System.out.println("5 + 20 = " + calculator.calculate(5, 20));

        calculator.setOperationStrategy(new SubtractionOperationStrategy());
        System.out.println("20 - 5 = " + calculator.calculate(20, 5));

        calculator.setOperationStrategy(new MultiplicationOperationStrategy());
        System.out.println("5 * 20 = " + calculator.calculate(5, 20));

        calculator.setOperationStrategy(new DivisionOperationStrategy());
        System.out.println("20 / 5 = " + calculator.calculate(20, 5));
    }
}

```

Line by line:

- `package com.design.patterns;` — Declares that this class belongs to the root package `com.design.patterns`. Spring Boot's default component-scan starts from the package of the `@SpringBootApplication`-annotated class, so placing the entry point at the root package ensures all sub-packages (`context`, `strategy`, `strategy.concreteStrategy`) are automatically scanned for Spring components.
- *(blank line)* — Standard separator between the package declaration and the import block.
- `import org.springframework.boot.SpringApplication;` — Brings the `SpringApplication` utility class into scope. `SpringApplication.run()` is the standard Spring Boot bootstrap method; without this import the call on the corresponding line would not compile.
- `import org.springframework.boot.autoconfigure.SpringBootApplication;` — Brings the `@SpringBootApplication` meta-annotation into scope. Without this import the annotation on the class declaration would be unresolvable at compile time.
- *(blank line)* — Blank line between the framework imports and the application-specific imports. Separating import groups by origin (framework vs. application) is a common Java style convention that improves readability.
- `import com.design.patterns.context.Calculator;` — Brings the `Calculator` context class into scope so it can be instantiated inside `main()`.
- `import com.design.patterns.strategy.concreteStrategy.AdditionOperationStrategy;` — Brings the concrete addition strategy into scope.
- `import com.design.patterns.strategy.concreteStrategy.DivisionOperationStrategy;` — Brings the concrete division strategy into scope.
- `import com.design.patterns.strategy.concreteStrategy.MultiplicationOperationStrategy;` — Brings the concrete multiplication strategy into scope.
- `import com.design.patterns.strategy.concreteStrategy.SubtractionOperationStrategy;` — Brings the concrete subtraction strategy into scope. Together, these four imports are the only place in the entire project where concrete strategy classes are referenced by name from outside their own files. Everywhere else — most importantly inside `Calculator` — the code depends only on the `IOperationStrategy` interface. This is intentional: the client is precisely the one place where a specific algorithm is chosen, so the concrete type names surface only here. The imports are listed alphabetically, a common IDE-enforced convention that keeps multi-import blocks scannable.
- *(blank line)* — Separates the import block from the class declaration.
- `@SpringBootApplication` — A Spring Boot meta-annotation that is shorthand for three annotations combined: `@Configuration` (this class can declare Spring beans), `@EnableAutoConfiguration` (Spring Boot automatically configures the application context based on the classpath), and `@ComponentScan` (Spring scans the `com.design.patterns` package and all sub-packages for annotated components). Without this annotation, the Spring context would not be bootstrapped. It is placed on the entry-point class because Spring Boot uses that class's package as the scan root.
- `public class StrategyDesignPattern {` — Declares the Spring Boot application class. Its name identifies exactly which design pattern this runnable module demonstrates, mirroring the naming convention used by every other module in this repository (and its `RealUseCase` sibling). It is `public` so that the JVM can invoke its `main` method as the application entry point. Opening `{` begins the class body.

- *(blank line)* — Blank line separating the class header from the method declaration.
- `public static void main(String[] args) {` — The standard Java entry-point method signature. `public` is required so the JVM can call it from outside the class. `static` is required so the JVM can invoke it without needing a pre-existing instance of the class. `void` is required because the JVM ignores any return value from `main`. `String[] args` receives command-line arguments passed when the JVM is launched; Spring Boot uses these to allow property overrides. The opening `{` begins the method body.
- `SpringApplication.run(StrategyDesignPattern.class, args);` — Starts the Spring Boot application. `StrategyDesignPattern.class` is the primary source class: Spring Boot uses its package as the component-scan root and its `@SpringBootApplication` annotation to drive auto-configuration. `args` passes command-line arguments through to Spring, allowing properties such as `server.port` to be overridden at launch. This line starts an embedded Tomcat server (because `spring-boot-starter-web` is on the classpath), initialises the application context, and writes the Spring Boot ASCII banner plus INFO startup log lines to standard output. After this call returns, the embedded server continues running on background threads.
- *(blank line)* — Blank line separating the Spring Boot bootstrap call from the pattern demonstration code. This visual separation makes clear that the lines below are the pattern demo, independent of the Spring context.
- `Calculator calculator = new Calculator(new AdditionOperationStrategy());` — Demonstrates the Strategy pattern's construction-time form. `new AdditionOperationStrategy()` creates a concrete strategy instance (an object with no fields). That instance is immediately passed into `new Calculator(...)` via constructor injection. The `calculator` local variable holds the fully configured context object, initially wired for addition.
- `System.out.println("5 + 20 = " + calculator.calculate(5, 20));` — Calls `calculate(5, 20)` on the context. Internally this chains to `operationStrategy.doOperation(5, 20)` inside `Calculator`, which dynamic-dispatches to `AdditionOperationStrategy.doOperation(5, 20)`, evaluating `5 + 20` and returning `25`. The `int` result `25` is concatenated onto the string `"5 + 20 = "` (Java auto-converts the `int` via `String.valueOf` during `+` concatenation) and the combined string is printed with a trailing newline.
- *(blank line)* — Blank line visually separating the addition demonstration from the subtraction demonstration.
- `calculator.setOperationStrategy(new SubtractionOperationStrategy());` — Demonstrates the Strategy pattern's *runtime* form. Rather than constructing a new `Calculator`, the existing `calculator` reference is re-armed in place: `new SubtractionOperationStrategy()` creates a fresh concrete strategy instance, and `setOperationStrategy` overwrites the context's internal `operationStrategy` field with it. The `calculator` object's identity is unchanged; only its behaviour changes.
- `System.out.println("20 - 5 = " + calculator.calculate(20, 5));` — Calls `calculate(20, 5)` on the same `calculator` instance. This now dispatches to `SubtractionOperationStrategy.doOperation(20, 5)`, evaluating `20 - 5` and returning `15`. The operand order is deliberately `(20, 5)` — not `(5, 20)` as in the addition call — because subtraction is not commutative and `20 - 5` is the meaningful reading. The result is concatenated and printed the same way as before.
- *(blank line)* — Blank line separating the subtraction demonstration from the multiplication demonstration.
- `calculator.setOperationStrategy(new MultiplicationOperationStrategy());` — Re-arms `calculator` a second time, now with a `MultiplicationOperationStrategy` instance, using the exact same mechanism as the subtraction swap above.
- `System.out.println("5 * 20 = " + calculator.calculate(5, 20));` — Calls `calculate(5, 20)`, dispatching to `MultiplicationOperationStrategy.doOperation(5, 20)`, which evaluates `5 * 20` and returns `100`. Multiplication is commutative, so the operand order here matches the addition call rather than the subtraction call; it is printed as `"5 * 20 = 100"`.
- *(blank line)* — Blank line separating the multiplication demonstration from the division demonstration.
- `calculator.setOperationStrategy(new DivisionOperationStrategy());` — Re-arms `calculator` a third time, now with a `DivisionOperationStrategy` instance.
- `System.out.println("20 / 5 = " + calculator.calculate(20, 5));` — Calls `calculate(20, 5)`, dispatching to `DivisionOperationStrategy.doOperation(20, 5)`, which evaluates the integer division `20 / 5` and returns `4`. As with subtraction, division is not commutative, so the operands are ordered `(20, 5)` to read as the intended "twenty divided by five".
- `}` — Closes the `main` method body. By this point, one `Calculator` instance has been swapped through four different strategies and used four times, which is the core proof that the pattern decouples "what algorithm runs" from "which object runs it."
- *(blank line)* — Blank line before the class closing brace.
- `}` — Closes the `StrategyDesignPattern` class body.

Why these design decisions

Why an interface, not an abstract class? An interface enforces a pure contract with no shared state or partial implementation. Any class — regardless of its own inheritance hierarchy — can implement `IOperationStrategy`. If an abstract class had been used, every concrete strategy would be forced to extend it, consuming Java's single inheritance slot and coupling all strategies to a common ancestor that adds no real value.

Why both constructor injection and a `setOperationStrategy` setter on `Calculator`? Constructor injection makes the dependency mandatory and visible at compile time: you cannot instantiate a `Calculator` without supplying an initial strategy, which prevents the field from ever starting out `null`. The setter is what turns the demo from "pick an algorithm once" into "swap the algorithm live, on one long-lived context object" — which is the scenario the Strategy pattern is really designed for (a pocket calculator's mode dial, a payment processor's selected gateway, a compressor's chosen codec — the object doing the work outlives any single algorithm choice). Because the field always starts populated via the constructor and the setter can only ever replace it with another valid `IOperationStrategy`, `operationStrategy` can never be observed as `null` from `calculate(...)`.

Why does `Calculator` depend on `IOperationStrategy` and not on a specific class? This is the Dependency Inversion Principle in action. `Calculator` depends on an abstraction, not a concretion. This means `Calculator` is closed for modification and open for extension: adding a new operation (e.g., `ModuloOperationStrategy`) requires writing one new class and changing zero existing classes — the Open/Closed Principle. That new class could be constructed at startup or swapped in later via `setOperationStrategy` with no change to `Calculator` either way.

Why are the concrete strategies in a sub-package (`concreteStrategy`) separate from the interface? The interface (`IOperationStrategy`) is what the context and any other consumer depend on. The concrete implementations are interchangeable details. Placing them in a sub-package makes the dependency direction visible in the package structure: `context` imports `strategy` (the abstraction), never `strategy.concreteStrategy` (the concretions).

Why does `main()` call `SpringApplication.run()` if the pattern demo does not use Spring? This project scaffolds the Spring Boot application runner as a standard container so the pattern can later be wired as a real Spring service (for example, injecting strategies as `@Component` beans selected by a `@Qualifier`). The pattern demonstration code that follows the `run()` call intentionally uses plain `new` instantiation and direct calls to `setOperationStrategy` to keep the core mechanism visible and free of framework detail.

Trade-offs:

Trade-off	Impact
Adding a new operation requires a new class file	More files than a single <code>if/switch</code> block, but each is tiny, independently testable, and adding one never touches existing code
The context always needs a strategy injected at construction, even though it can be swapped later	Slightly more setup for the caller, but eliminates invalid-null-state bugs — the field is never <code>null</code> at any point in the object's lifetime
Integer division truncates silently	Acceptable for a pedagogical example; production code would use <code>double</code> or <code>BigDecimal</code> and guard against division by zero
<code>IOperationStrategy</code> is a functional interface	Can be replaced by a lambda (<code>(x, y) -> x + y</code>), which is a strength — lambdas are lightweight strategies

Execution flow (step-by-step trace of what happens when `main()` runs)

1. The JVM invokes `StrategyDesignPattern.main(args)`.
2. `SpringApplication.run(StrategyDesignPattern.class, args)` is called.
3. Spring Boot initialises an `AnnotationConfigServletWebServerApplicationContext`.
4. The embedded Tomcat web server starts on port 8080 (default).
5. Spring Boot writes its ASCII-art banner and INFO startup log lines to standard output.
6. The log line `Started StrategyDesignPattern in X.XXX seconds (JVM running for X.XXX)` appears.
7. `run()` returns after the context is fully refreshed; the Tomcat server remains alive on non-daemon background threads.
8. Execution resumes in `main()` after the `run()` call.
9. `new AdditionOperationStrategy()` constructs an instance of the addition strategy — a plain Java object with no fields and no initialisation logic. `new Calculator(additionStrategy)` constructs a `Calculator`; inside its constructor, `this.operationStrategy = operationStrategy` stores that reference into the field.
10. `calculator.calculate(5, 20)` is invoked. Inside `calculate`, `operationStrategy.doOperation(5, 20)` is called on the field. The JVM uses dynamic dispatch: the runtime type of `operationStrategy` is `AdditionOperationStrategy`, so `AdditionOperationStrategy.doOperation` runs, evaluates `5 + 20`, and returns `25`. `System.out.println("5 + 20 = " + 25)` writes `5 + 20 = 25` to standard output.
11. `calculator.setOperationStrategy(new SubtractionOperationStrategy())` overwrites the same `calculator` object's `operationStrategy` field with a new `SubtractionOperationStrategy` instance. The `calculator` reference itself does not change.
12. `calculator.calculate(20, 5)` is invoked. Dynamic dispatch now resolves to `SubtractionOperationStrategy.doOperation`, which evaluates `20 - 5` and returns `15`. `System.out.println("20 - 5 = " + 15)` writes `20 - 5 = 15`.
13. `calculator.setOperationStrategy(new MultiplicationOperationStrategy())` overwrites the field again. `calculator.calculate(5, 20)` dispatches to `MultiplicationOperationStrategy.doOperation`, evaluates `5 * 20`, returns `100`, and `5 * 20 = 100` is printed.

14. `calculator.setOperationStrategy(new DivisionOperationStrategy())` overwrites the field a third time. `calculator.calculate(20, 5)` dispatches to `DivisionOperationStrategy.doOperation`, evaluates the integer division $20 / 5$, returns 4, and $20 / 5 = 4$ is printed.
15. `main()` returns; the JVM remains alive because the Spring Boot embedded Tomcat server holds live non-daemon threads, until the process is stopped externally (Ctrl-C or a timeout).

Expected output

Spring Boot's startup banner and INFO log lines appear first (exact content, PID, and timings vary by run and by machine). The four lines produced by the pattern demonstration itself follow immediately after the startup sequence. This is the actual, verbatim console output captured from a real run of the built fat JAR on this module (`java -jar target/Strategy_Design_Pattern-0.0.1-SNAPSHOT.jar`):

[illegible]

```

2026-07-10 11:45:07.754 INFO 3532647 --- [main] c.design.patterns.StrategyDesignPattern : Starting
StrategyDesignPattern v0.0.1-SNAPSHOT using Java 11.0.31 on gaian with PID 3532647
(/home/gaian/Videos/G04/Design-Pattern-G04/Behavioral/Strategy_Design_Pattern/target/Strategy_Design_Pattern-
0.0.1-SNAPSHOT.jar started by gaian in /home/gaian/Videos/G04/Design-Pattern-
G04/Behavioral/Strategy_Design_Pattern)
2026-07-10 11:45:07.763 INFO 3532647 --- [main] c.design.patterns.StrategyDesignPattern : No
active profile set, falling back to 1 default profile: "default"
2026-07-10 11:45:09.396 INFO 3532647 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat
initialized with port(s): 8080 (http)
2026-07-10 11:45:09.424 INFO 3532647 --- [main] o.apache.catalina.core.StandardService : Starting
service [Tomcat]
2026-07-10 11:45:09.425 INFO 3532647 --- [main] org.apache.catalina.core.StandardEngine : Starting
Servlet engine: [Apache Tomcat/9.0.70]
2026-07-10 11:45:09.588 INFO 3532647 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] :
Initializing Spring embedded WebApplicationContext
2026-07-10 11:45:09.589 INFO 3532647 --- [main] w.s.c.ServletWebServerApplicationContext : Root
WebApplicationContext: initialization completed in 1732 ms
2026-07-10 11:45:10.019 INFO 3532647 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat
started on port(s): 8080 (http) with context path ''
2026-07-10 11:45:10.033 INFO 3532647 --- [main] c.design.patterns.StrategyDesignPattern : Started
StrategyDesignPattern in 2.854 seconds (JVM running for 3.183)
5 + 20 = 25
20 - 5 = 15
5 * 20 = 100
20 / 5 = 4

```

The four key lines produced by the pattern demonstration are:

$$\begin{array}{l} 5 + 20 = 25 \\ 20 - 5 = 15 \\ 5 * 20 = 100 \\ 20 / 5 = 4 \end{array}$$

These are, respectively, the results of `AdditionOperationStrategy.doOperation(5, 20)`, `SubtractionOperationStrategy.doOperation(20, 5)`, `MultiplicationOperationStrategy.doOperation(5, 20)`, and `DivisionOperationStrategy.doOperation(20, 5)`, each printed immediately after `calculator.setOperationStrategy(...)` re-armed the same `Calculator` instance with a new algorithm. After the last line prints, the process keeps running (embedded Tomcat holds non-daemon threads); stop it with Ctrl-C or a timeout.

How to run

From the module directory (Behavioral/Strategy Design Pattern/):

```
# Build the fat JAR
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean package -DskipTests

# Run via the Spring Boot fat JAR
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -jar target/Strategy Design Pattern-0.0.1-SNAPSHOT.jar
```

From the repository root (builds the module and its parent POM):

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean package -pl Behavioral/Strategy_Design_Pattern -am -DskipTests
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -jar
Behavioral/Strategy_Design_Pattern/target/Strategy_Design_Pattern-0.0.1-SNAPSHOT.jar
```

Alternatively, compile only and run directly off the classes directory with the dependency classpath (no fat jar needed):

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -cp "target/classes:$(mvn -o dependency:build-classpath -Dmdep.outputFile=/dev/stdout -q)\" \
    com.design.patterns.StrategyDesignPattern
```

The `-o` (offline) flag works once the parent reactor and dependencies are already in the local Maven cache; drop it on a first build. `JAVA_HOME` must point at a JDK 11 installation for this reactor.

Main class fully qualified name (for reference):

```
com.design.patterns.StrategyDesignPattern
```

Other real-world problems this pattern can solve

Domain	Context class	Interchangeable strategies
Payment processing	PaymentProcessor	CreditCardStrategy , PayPalStrategy , CryptoStrategy
File compression	Archiver	ZipStrategy , GzipStrategy , Bzip2Strategy
Sorting	DataSorter	QuickSortStrategy , MergeSortStrategy , TimSortStrategy
Authentication	AuthService	JwtStrategy , OAuth2Strategy , LdapStrategy
Route navigation	Navigator	FastestRouteStrategy , ShortestRouteStrategy , EcoRouteStrategy
Logging output	Logger	ConsoleLogStrategy , FileLogStrategy , RemoteSyslogStrategy
Tax calculation	Invoice	USTaxStrategy , EUVatStrategy , TaxExemptStrategy
Pricing discounts	PriceCalculator	SeasonalDiscountStrategy , LoyaltyDiscountStrategy , NoDiscountStrategy
Text rendering	DocumentRenderer	HtmlRenderStrategy , PdfRenderStrategy , PlainTextStrategy

In every case the context class remains unchanged; only the strategy passed in at construction (or injected by a DI container such as Spring) determines the runtime behaviour.